MetaMask – Delegator Safe Module

1 Executive Summary

2 Scope

2.1 Objectives

3 System Overview

4 Architectural Incompatibilities and Friction Points

5 Findings

5.1 Missing Delegation Management and Self-Authorization Incompatibility Major ✓ Fixed

5.2 Interface-Behavior Misalignment in `executeFromExecutor()` Medium ✓ Fixed

5.3 Safe Owner's Private Key Not Needed in Deployment Script Medium ✓ Fixed

5.4 Dependencies Using Unreleased Versions Minor ✓ Fixed

5.5 Deployment of a Proxy Could Be Frontun Minor ✓ Fixed

5.6 `_execute` Doesn't Propagate Original Revert Reason Minor ✓ Fixed

5.7 Misleading Behavior of Calls Made Directly on the Implementation Contract Minor ✓ Fixed

5.8 Solidity Version Pragma Allows Outdated Compiler Versions Minor ✓ Fixed

5.9 Module Emits No Events ✓ Fixed

5.10 Code Triggers Some Compiler Warnings ✓ Fixed

Appendix 1 - Files in Scope

Appendix 2 - Disclosure

A.2.1 Purpose of Reports

A.2.2 Links to Other Web Sites from This Web Site

A.2.3 Timeliness of Content

Date	November 2025
Auditors	Heiko Fisch, Martin Ortner

1 Executive Summary

This report presents the results of our engagement with **MetaMask** to review the new **Delegator Safe Module**.

The review was conducted in the week of **November 3–7, 2025**, with a total effort of 5 person-days.

The MetaMask Delegator Safe Module is a minimal Safe module that adds delegation capabilities via the MetaMask Delegation Framework to Safe Smart Accounts. The module is designed to receive execution requests exclusively from a trusted Delegation Manager contract, which is responsible for the prior validation of delegation chains. It supports single and batch transaction execution and relies on the Safe for signature verification. A system overview is given in section 3, highlighting the main aspects of the module, the module factory, and the surrounding components Safe and Delegation Manager.

The findings are discussed in section 5. Most of them are minor or informational, and two issues of medium severity were identified. Our only major concern is the missing possibility to disable delegations. This security-relevant feature is provided by the Delegation Framework's `DelegationManager` contract, but the module implements no way to utilize it. This can be addressed by adding passthrough functions for the related functionality to the module – or an alternative design without the need for passthrough functions could be explored.

The root cause of this problem (and the related ones we discuss in finding 5.1) is the separation of Safe / delegating account on the one hand and module / `delegator` in the root `Delegation` struct on the other hand. As there is no such distinction in regular DeleGator accounts, this separation can lead to further incompatibilities and friction points, and we believe that a revised architecture which avoids this separation might be the preferable solution.

However, the client has expressed that they're comfortable proceeding with the current design and has added passthrough functionality to the module to address finding 5.1 in general and allow the revocation of delegations in particular. Their rationale for this decision is outlined in the "Resolution" box for this issue. Our remaining concerns regarding the current architecture are summarized in section 4. This section has been added during the fix review phase and can be seen as an addendum to the discussion in finding 5.1 and the chosen solution. Apart from the points raised in this section, we consider all findings fixed in the final version we reviewed (b61e1e4).

2 Scope

This review focused initially on the following repository and code revision:

- MetaMask/delegator-safe-module at commit hash 77dae3ffe31df1786ce633a1b510d3c31f922bd5

The updated and finally reviewed version of the codebase is:

- MetaMask/delegator-safe-module at commit hash b61e1e4adad1526f970d6b39469e031b4e7ac2a2

A detailed list of files in scope, together with their SHA-1 hashes, can be found in the Appendix. The Safe and Delegation Framework code was not in scope for this review.

While we tried to ensure compatibility with the `DelegationManager` itself, reviewing the entire Delegation Framework code for compatibility was not possible in the short time we had for this review. In particular, compatibility with the enforcers that the framework provides was out of scope.

2.1 Objectives

Together with the client, we identified the following priorities for this review:

1. Correctness of the implementation, consistent with the intended functionality and without unintended edge cases.
2. Identify known vulnerabilities particular to smart contract systems, as outlined in our Smart Contract Best Practices, and the Smart Contract Weakness Classification Registry.
3. Correct integration with Safe and Delegation Manager.

3 System Overview

The MetaMask Delegator Safe Module enables the MetaMask Delegation Framework to work with Safe Smart Accounts. The involved components with their main functionalities are outlined below.

Delegator Module Factory

- `DelegatorModuleFactory` is deployed with address of trusted Delegation Manager, stored as immutable
- Deploys `DelegatorModule` implementation contract in its constructor with same, immutable Delegation Manager address

- Deploys deterministic clones using CREATE2 via Solady's `LibClone.cloneDeterministic()` (`LibClone.createDeterministicClone()` in final version (b61e1e4))
- Each clone is bound to a specific Safe address through immutable clone arguments

Delegator Module

Execution Capabilities:

- *Execution Function*: `executeFromExecutor(ModeCode _mode, bytes calldata _executionCalldata)`
- *Access Control*: Protected with `onlyDelegationManager` modifier
- *Mode Support*:
  - CallType: `CALLTYPE_SINGLE` (single transaction) and `CALLTYPE_BATCH` (multiple transactions)
  - ExecType: `EXECTYPE_DEFAULT` (revert on failure) only
- *Execution Flow*: Decodes mode and calldata using ERC-7579 libraries (`ModeLib`, `ExecutionLib`), then forwards to Safe's `execTransactionFromModuleReturnData()`
- *Return Handling*: Collects and returns all transaction return data; reverts in case of failure
- Added in final version (b61e1e4): `execute()` function to make arbitrary calls with module as `msg.sender`; only callable by Safe

Interface Implementations:

- *ERC-165*: `supportsInterface()` declares support for `IERC165`, `IDeleGatorCore`, and `IERC1271` interfaces
- *ERC-1271*: `isValidSignature()` delegates signature validation directly to the Safe contract
- *IDeleGatorCore*: Implements the Delegation Framework's core executor interface

Safe

Integration:

- The module integrates with Safe through the standard module interface
- Safe owners must explicitly enable the `DelegatorModule` via `enableModule()`
- Safe owners can disable the module at any time via `disableModule()`

Transaction Execution:

- *Interface Used*: `ISafe.execTransactionFromModuleReturnData(address to, uint256 value, bytes memory data, Enum.Operation operation)`
- *Operation Type*: Always uses `Enum.Operation.Call` (standard call, not delegatecall)
- *Error Handling*: Safe returns `(bool success, bytes memory returnData)`; module reverts with `ExecutionFailed()` if `success == false`; in final version (b61e1e4), revert reason is bubbled up

Delegation Manager

Expected Responsibilities:

- Check validity of delegation chains
- This involves calling `isValidSignature()` on the module
- Call `executeFromExecutor()` on root delegator (the module, in our case) if and only if delegation chain valid

Security Dependencies:

- Module provides no validation beyond caller check
- All delegation security logic must be implemented in `DelegationManager`
- `DelegationManager` has full control over Safe through module
- Safe owners must understand delegations and carefully review them before signing

4 Architectural Incompatibilities and Friction Points

The architecture chosen by the Delegator team to bring delegation capabilities to Safes is that of a pure module. This means that there is a separate contract – the module – which the Safe trusts and which can instruct the Safe to execute transactions. Crucially, it is the module's address that has to be the `delegator` in the root `Delegation`, not the Safe itself, whereas for "regular" DeleGator accounts, the `delegator` *is* the delegating account. This separation between two different entities – Safe / delegating account on the one hand and module / `delegator` in the root `Delegation` on the other hand – has the potential to create incompatibilities because it doesn't exist for DeleGator accounts.

We have identified examples of such incompatibilites in the delegation framework's `DelegationManager` and discuss these in finding 5.1. Specifically, delegations can be disabled by calling the `disableDelegation()` function on the `DelegationManager`, but the `msg.sender` must be the `delegator` of the corresponding `Delegation`. In the initial version we reviewed (77dae3f), the module didn't have the capability to make such calls. This has been addressed in the final version (b61e1e4) by adding a general call forwarding functionality to the module that allows the Safe (and only the Safe) to make arbitrary calls with the module as `msg.sender`. This alleviates the problem at hand; the Safe can now disable delegations. But it should be noted that this has to be done in a different way: A regular DeleGator account would directly call `disableDelegation()` on the `DelegationManager`, whereas the Safe has to make a call to the module – which then calls `disableDelegation()` itself.

This difference is not only relevant if a delegation is disabled *directly*, but also for the *delegated* revocation of a delegation. In this case, the redeemer would have to construct different transaction calldata, depending on whether the delegating account is a DeleGator or a Safe. The same is true for `enableDelegation()` and (at least in some cases) `redeemDelegations()`, but we won't discuss these in detail here. For the purposes of this discussion, suffice it to say that we are in essentially the same situation as with `disableDelegation()`. More information on these two functions can be found in issue 5.1. The necessity to build calldata differently depending on the type of the delegating account – even if only in some situations – is a potential source of subtle bugs and unexpected problems, and it makes the task to construct correct calldata for redemptions more difficult and error-prone.

Another area of concern are incompatibilities with *enforcers*, contracts that can be added to a delegation and enforce certain conditions when the delegation is used. We describe two hypothetical enforcers that wouldn't work with the current design as a

pure module.

The first example is an enforcer that ensures that the delegating account has a certain minimum balance in a particular asset like ETH or an ERC-20 token. The delegating account is not directly accessible in a delegation, but since for "standard" `DeleGator` accounts, the root delegator is the same as the delegating account, it is conceivable that someone tries to implement such an enforcer by querying the balance of the `delegator`. If they test their code with a `DeleGator` account, they'll find it works, but in the case of a Safe, this enforcer checks the module's balance, not the Safe's, and can therefore fail in both directions: It could let a transaction pass that it is supposed to stop or stop a transaction that should pass. The former is often the more problematic case. Two considerations limit the relevance of this example: (1) It seems unlikely that the module has a non-zero balance in the first place, thus making the more dangerous situation unlikely as well. (2) The problem is easy to fix or avoid in the first place by providing the address whose balance is to be checked as enforcer *term*, i.e., as delegator-chosen parameter. Nevertheless, we think that it is at least conceivable that someone implements an enforcer which assumes the root `delegator` is the delegating account, especially as this works with DeleGator accounts, and that this enforcer is then used with Safes despite being incompatible.

A perhaps more plausible but also more "selected" example is an enforcer that only lets transactions pass which disable delegations. The natural way to implement such an enforcer is to allow only calls to the `DelegationManager` and to restrict these to the `disableDelegation()` function. That works for a regular DeleGator account, but as already discussed above, a Safe can't disable delegations without going through the module. The enforcer wouldn't let the call to the module pass, and revoking the delegation would fail. While unintended failing is the frequently less critical direction, in this case, it could be a fairly serious problem if the delegate is, for example, a trusted monitoring service that is supposed to disable delegations under certain conditions as a protective measure.

Both examples are hypothetical, but since the time for this review was not sufficient to examine all enforcers in the delegation framework for compatibility, we can't rule out that real, existing enforcers which work for DeleGator accounts break for Safes. Naturally, this could also be the case for enforcers developed by third parties.

In conclusion, we see, at the very least, shortcomings and friction points of this design and recommend to users and developers alike to understand and be mindful of the distinction between Safe / delegating account on the one hand and module / `delegator` on the other and of the implications of this distinction for code and calldata. To support this understanding, the client should carefully document and explain these points. Incompatible enforcers should be identified and fixed to also work with Safes, or the lack of compatibility should be clearly documented. The same is true for code that prepares redemption calldata.

As a cleaner solution, we still recommend the exploration of alternative architectures, possibly as a combined fallback handler and module, to make delegation-enabled Safes behave more like regular DeleGator accounts and avoid incompatibilities in the first place.

5 Findings

Each issue has an assigned severity:

- Critical issues are directly exploitable security vulnerabilities that need to be fixed.
- Major issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Medium issues are objective in nature but are not security vulnerabilities. These should be addressed unless there is a clear reason not to.
- Minor issues are subjective in nature. They are typically suggestions around best practices or readability. Code maintainers should use their own judgment as to whether to address such issues.
- Issues without a severity are general recommendations or optional improvements. They are not related to security or correctness and may be addressed at the discretion of the maintainer.

5.1 Missing Delegation Management and Self-Authorization Incompatibility Major ✓ Fixed

Resolution

The client has added a general `execute()` function to the module. It only accepts calls from the Safe and allows the latter to initiate arbitrary calls with the module as `msg.sender`. This addresses all passthrough needs mentioned below, as well as others that could be useful now (such as the ability to increment the nonce in `NonceEnforcer`) or later (with changes/additions in `DelegationManager` or with new enforcers). In particular, it is now possible to disable delegations, which was the most important problem identified in this finding.

As mentioned below, the design as a pure module entails that there are two addresses involved: the Safe – the actual account that's delegating – and the module, which acts as a middle man between Safe and Delegation Manager and has to be the `delegator` in a `Delegation` struct instead of the Safe itself. This is in contrast to a regular DeleGator account, which doesn't require such a middle man and is used directly as `delegator`.

As this separation between two different entities is the root cause of this issue and can lead to further incompatibilities and friction points, we suggested exploring different architectural options, such as a combined fallback handler and module, which would remove the middle man and make the Safe the only entity to engage with.

While the client acknowledges that occasional confusion and errors might occur, they have expressed that they're comfortable proceeding with the current approach. Moreover, the general `execute()` function on the module somewhat mitigates the situation; for instance, if tokens are accidentally sent to the module instead of the Safe, they can still be retrieved or used by the Safe via `execute()`. The Delegator team also considers adding more documentation, tutorials, possibly an SDK, etc. to further reduce the risk of wrong usage.

Nevertheless, we believe that problematic aspects of the current architecture remain. As the points raised in this finding have been resolved with the client's changes, we set it to "Fixed" and discuss our remaining concerns in the new section 4 above.

Description

With the chosen design, the `delegator` in a `Delegation` struct has to be the module, not the Safe, because only the module implements the `executeFromExecutor()` function. This leads to the following two problems:

A. Missing Delegation Management
Safe owners cannot call `disableDelegation()` or `enableDelegation()` in the (official) `DelegationManager` through the module, simply because there is no code to support this. While the Safe could technically call these functions directly, the `DelegationManager` treats the module as the delegator, not the Safe, as discussed above. As a result, calls from the Safe do not satisfy the `msg.sender == delegator` requirement, and delegations cannot be revoked or managed on behalf of the module.

`DelegationManager.sol:L85-L95`

```
/**
 * @notice This method is used to disable a delegation. Disabled delegations will fail upon redemption.
 * @dev This method MUST be called by the delegator
 * @param _delegation The delegation to be disabled
 */
function disableDelegation(Delegation calldata _delegation) external onlyDeleGator(_delegation.delegator) {
    bytes32 delegationHash_ = getDelegationHash(_delegation);
    if (disabledDelegations[delegationHash_]) revert AlreadyDisabled();
    disabledDelegations[delegationHash_] = true;
    emit DisabledDelegation(delegationHash_, _delegation.delegator, _delegation.delegate, _delegation);
}
```

`DelegationManager.sol:L48-L56`

```
 * @notice Require the caller to be the delegator
 * This is to prevent others from accessing protected methods.
 * @dev Check that the caller is delegator.
 */
modifier onlyDeleGator(address delegator) {
    if (delegator != msg.sender) revert InvalidDelegator();
    _;
}
```

The ability to disable delegations is an important security feature of the Delegation Framework and should also be supported by this module.

B. Self-Authorization Incompatibility
The `DelegationManager` implements a shortcut, where redeeming an empty array of delegations triggers a call to `executeFromExecutor()` on `msg.sender`:

`DelegationManager.sol:L144-L147`

```
if (delegations_.length == 0) {
    // Special case: If the permissionContext is empty, treat it as a self authorized execution
    batchDelegations_[batchIndex_] = new Delegation[](0);
    batchDelegationHashes_[batchIndex_] = new bytes32[](0);
```

`DelegationManager.sol:L230-L232`

```
if (batchDelegations_[batchIndex_].length == 0) {
    // Special case: If there are no delegations, defer the call to the caller.
    IDeleGatorCore(msg.sender).executeFromExecutor(_modes[batchIndex_], _executionCallDatas[batchIndex_]);
```

This shortcut does not work for Safes. When the Safe tries to redeem, the `DelegationManager` calls `msg.sender`, but the Safe does not implement `executeFromExecutor()`. The module does, but it has no mechanism to call `redeemDelegations()` on the Delegation Manager.

This is certainly less problematic than A, but it is unclear how much of an issue it is. On the one hand, calling `redeemDelegations()` isn't supposed to require a DeleGator account; any smart contract account and even an EOA can redeem. On the other hand, the module is supposed to add DeleGator capabilites to a Safe, so having this shortcut not available for Safes might be perceived as a limitation.

Recommendation

Consider adding passthrough functions in `DelegatorModule` to allow Safe owners to manage delegations and to support redeeming with an empty delegation array. More specifically:

A. Add `disableDelegation()` and `enableDelegation()` functions to `DelegatorModule`. Both must only accept calls from the module's Safe and forward the request to the Delegation Manager. Add clarifying documentation that a Safe can't call these functions directly on `DelegationManager`, but has to take a detour through the module.

B. Similarly, consider adding a `redeemDelegations()` function to `DelegatorModule` that only accepts calls from the module's Safe and forwards the request to the Delegation Manager. Add clarifying documentation that, while a Safe *can* call `redeemDelegations()` directly on `DelegationManager`, the shortcut with an empty delegation array works only if the call is routed through the module. Alternatively, if the shortcut is not considered important and a `redeemDelegations()` function not added to the module, it should be documented that redeeming with an empty delegations array doesn't work.

We'd like to conclude this discussion with the following two remarks:

1. This finding assumes that the module is used with the official `DelegationManager`. If the code is ever to be used with an alternative Delegation Manager or if the standard Delegation Manager receives an upgrade, compatibility of this module has to be reassessed. The same is true, of course, if Safe is changed in a way that affects modules, but we consider the Safe module interface more stable than `DelegationManager`.

2. The need for these passthrough functions is a consequence of the implementation as a pure module. Essentially, the module acts as middle man between Safe and Delegation Manager. A design as combined fallback handler and module would allow the Safe itself to be the `delegator` and remove the middle man. While each design comes with its own set of advantages and disadvantages, the necessity to add these passthrough functions suggests that a reevaluation of the design choice and an exploration of alternatives might be worthwhile.

5.2 Interface-Behavior Misalignment in `executeFromExecutor()` Medium ✓ Fixed

Resolution
Function `executeFromExecutor()` now reverts if `msg.value` is non-zero.

Description

The `executeFromExecutor()` function is marked as `payable`, but it completely ignores the ETH it could potentially receive:

src/DelegatorModule.sol:L73-L103

```
function executeFromExecutor(
    ModeCode _mode,
    bytes calldata _executionCalldata
)
    external
    payable
    onlyDelegationManager
    returns (bytes[] memory returnData_)
{
    (CallType callType_, ExecType execType_,,) = _mode.decode();

    // Check if calltype is batch or single
    if (callType_ == CALLTYPE_BATCH) {
        // Destructure executionCallData according to batched exec
        Execution[] calldata executions_ = _executionCalldata.decodeBatch();
        // check if execType is revert or try
        if (execType_ == EXECTYPE_DEFAULT) returnData_ = _execute(executions_);
        else revert UnsupportedExecType(execType_);
    } else if (callType_ == CALLTYPE_SINGLE) {
        // Destructure executionCallData according to single exec
        (address target_, uint256 value_, bytes calldata callData_) = _executionCalldata.decodeSingle();
        returnData_ = new bytes[](1);
        if (execType_ == EXECTYPE_DEFAULT) {
            returnData_[0] = _execute(target_, value_, callData_);
        } else {
            revert UnsupportedExecType(execType_);
        }
    } else {
        revert UnsupportedCallType(callType_);
    }
}
```

In fact, the entire contract has no functionality to deal with ETH received. This doesn't cause any immediate problems, though, because the official `DelegationManager` does not send ETH when calling `executeFromExecutor()`:

`DelegationManager.sol:L231-232`

```
// Special case: If there are no delegations, defer the call to the caller.
IDeleGatorCore(msg.sender).executeFromExecutor(_modes[batchIndex_], _executionCallDatas[batchIndex_]);
```

`DelegationManager.sol:L251-L254`

```
// Perform execution
IDeleGatorCore(batchDelegations_[batchIndex_][batchDelegations_[batchIndex_].length - 1].delegator)
    .executeFromExecutor(_modes[batchIndex_], _executionCallDatas[batchIndex_]);
```

Nevertheless, if the module is ever used with an alternative Delegation Manager that does send ETH in these calls, it will be stuck in the `DelegatorModule`.

It is not possible to simply remove the `payable` modifier from the function definition, as `executeFromExecutor()` is declared in the `IDeleGatorCore` interface from the Delegation Framework, and `DelegatorModule` inherits from this interface. We assume that the interface can't or shouldn't be changed.

```
interface IDeleGatorCore is IERC1271 {
    /**
     * @dev Executes a transaction on behalf of the account.
     *         This function is intended to be called by Executor Modules
     * @dev Ensure adequate authorization control: i.e. onlyExecutorModule
     * @dev If a mode is requested that is not supported by the Account, it MUST revert
     * @dev Related: @erc7579/MSAAdvanced.sol
     * @param _mode The encoded execution mode of the transaction. See @erc7579/ModeLib.sol for details.
     * @param _executionCalldata The encoded execution call data
     */
    function executeFromExecutor(
        ModeCode _mode,
        bytes calldata _executionCalldata
    )
        external
        payable
        returns (bytes[] memory returnData);
}
```

Recommendation

The development team should clarify the intended behavior of the module in case ETH is ever sent and choose one of the two following solutions:

1. In `executeFromExecutor()`, revert if `msg.value` is non-zero.
2. Add handling logic to properly forward or manage any received ETH (e.g., transferring it to the Safe) if value-bearing calls are expected.

Remark

As a related point, `executeFromExecutor()` diligently collects the return values from the underlying calls and passes them back to the Delegation Manager. However, these are ignored by the standard `DelegationManager`, as can be seen in the excerpts above.

5.3 Safe Owner's Private Key Not Needed in Deployment Script Medium ✓ Fixed

Resolution

The environment variable `SAFE_OWNER_PRIVATE_KEY` has been renamed to `DEPLOYER_PRIVATE_KEY`, thus no longer suggesting that the Safe owner's private key is needed to deploy the module. Following Foundry's advice for the use of private keys in scripts, a one-time key should be used for deployment.

Description

The deployment script loads `SAFE_OWNER_PRIVATE_KEY` from the environment and uses it to broadcast the deployment transaction via `vm.startBroadcast(safeOwnerPrivateKey)`:

script/DeployDelegatorModule.s.sol:L22-L25

```
uint256 safeOwnerPrivateKey = vm.envUint("SAFE_OWNER_PRIVATE_KEY");
address factoryAddress = vm.envOr("FACTORY_ADDRESS", address(0));
// Start broadcast for deployment transaction
vm.startBroadcast(safeOwnerPrivateKey);
```

However, using the Safe owner's private key to broadcast deployment transactions is not necessary and introduces security risk. Deploying the factory (if necessary) and a module proxy for the Safe does not require Safe owner privileges – anyone can do that. The Safe owner's signature is only required for the subsequent `enableModule()` transaction on the Safe contract, but that has to be done in a separate step after deployment:

script/DeployDelegatorModule.s.sol:L48-L53

```
console2.log("****************************************");
console2.log("Enable the module in the Safe UI using the transaction builder.");
console2.log("set the contract to call to:", safeAddress);
console2.log("set the method to call to: enableModule");
console2.log("set the moduleAddress parameter to:", deployedModule);
```

This might very well be considered a naming issue. In fact, using an arbitrary private key instead of the Safe owner's works just fine and, moreover, the README mentions a `DEPLOYER_PRIVATE_KEY` instead of a `SAFE_OWNER_PRIVATE_KEY`, although an environment variable of that name is not used in the deployment script:

README.md?plain=1:L61

```
- `DEPLOYER_PRIVATE_KEY`: Private key deployer
```

In any case, the name might lead users to expose their Safe owner's private key unnecessarily, so – even if a wrong variable name is at the heart of this – it creates a security risk indeed.

Recommendation

Remove the `SAFE_OWNER_PRIVATE_KEY` parameter and use a standard deployment procedure that doesn't require (and doesn't suggest that it requires) Safe-related privileges. The Safe owner's private key can be reserved exclusively for the `enableModule()`

transaction that must be executed on the Safe contract after deployment; it is not needed in the deployment script. The README should be adapted accordingly.

5.4 Dependencies Using Unreleased Versions Minor ✓ Fixed

Resolution

The dependencies have been updated as follows:

- delegation-framework @ bfbdf97, which is v1.3.0,
- forge-std @ 8e40513, which is v1.11.0,
- safe-smart-account @ dc437e8, which is v1.5.0,
- solady @ acd959a, which is v0.1.26.

Description

The project's dependencies appear to be installed with `forge install` using unreleased versions, at seemingly random and sometimes fairly old commits. Unreleased revisions of a codebase might very well reflect work in progress and have bugs and/or inconsistencies. And older versions, released or not, frequently have issues that have become known later.

Recommendation

Dependencies should be installed in released versions without known issues.

5.5 Deployment of a Proxy Could Be Frontun Minor ✓ Fixed

Resolution

The `deploy()` function in the factory now uses `LibClone.createDeterministicClone()`, which – unlike the previously used `LibClone.cloneDeterministic()` – doesn't revert if the proxy is already deployed. Instead, it returns a boolean value in addition to the proxy address, indicating whether deployment was necessary or not. The factory's `deploy()` function returns the same two values (in reversed order, though) and also emits them (in addition to other data) in the `ModuleDeployed()` event. This allows onchain callers and offchain observers to determine whether the module was already deployed or not and to react accordingly. The deployment script utilizes the boolean return value to show a more appropriate message in case the module was already deployed. Moreover, a NatSpec comment has been added which clarifies that `SALT` should be "shorter than 32 bytes".

Description

The `deploy()` function in `DelegatorModuleFactory` takes as input a Safe address and a salt and deploys a proxy via `LibClone.cloneDeterministic()`:

src/DelegatorModuleFactory.sol:L24-L32

```
/// @notice Deploys a DelegatorModule clone for a given safe
/// @param safe The address of the Safe contract
/// @param salt The salt for CREATE2
/// @return module The address of the deployed module
function deploy(address safe, bytes32 salt) external returns (address module) {
    bytes memory args = abi.encodePacked(safe); // 20 bytes
    module = LibClone.cloneDeterministic(implementation, args, salt);
    emit ModuleDeployed(safe, implementation, module);
}
```

If a user wants to deploy a module (proxy) for their Safe, they call `deploy()` with the Safe's address and a chosen salt, which is passed into `create2()`. It is unclear whether the salt has any particular meaning attached to it and should go into `create2()` unaltered or whether the salt is only there to allow different proxy deployments for the same Safe and the specific value used in `create2()` doesn't matter.

Consider the following situation: Alice tries to deploy her proxy calling `deploy()`. Bob frontruns Alice's `deploy()` transaction with the same (Alice's) Safe address and the same salt. No real harm would come out of this for Alice; she could simply use the proxy that Bob deployed. However, her transaction would revert, possibly leading to some irritation, and it would require some investigation on her part to figure out what went wrong.

Recommendation

If we assume the actual salt value doesn't really matter, a simple way to address this problem is to hash the originally given `salt` together with `msg.sender` and use this as the salt passed to `LibClone.cloneDeterministic()` and, ultimately, to `create2()`. Bob will then be unable to interfere with Alice's deployment. Apart from the altered salt, another possible downside is that Alice can only deploy her proxy to the same address on another chain if she controls the sender address there too.

Hence, a more graceful way of dealing with this potential issue is to first check if the predicted address for the clone already has code and try the deployment only if that's not the case. This approach leverages the fact that it doesn't matter who deploys Alice's proxy.

Remark

`DeployDelegatorModule.s.sol` reads a `SALT` string from the environment and explicitly converts the `bytes` from `abi.encodePacked()` to `bytes32`:

script/DeployDelegatorModule.s.sol:L17

```
bytes32 salt = bytes32(abi.encodePacked(vm.envString("SALT")));
```

Hence, any `SALT` longer than 32 bytes will be truncated. Depending on whether strings longer than 32 bytes should be supported and whether the particular `SALT` can or mustn't be altered, applying a hash function might be more appropriate than truncating the input.

5.6 `_execute` Doesn't Propagate Original Revert Reason Minor ✓ Fixed

Resolution

A `_checkSuccess()` function has been implemented, which

- bubbles up the revert reason if the failing Safe transaction provides one and
- throws the custom `ExecutionFailed()` error if the Safe transaction fails with empty return data.

The assembly block in `_checkSuccess()` is memory-safe and should be annotated accordingly.

Description

In the `DelegatorModule` contract, the `_execute()` function currently uses a custom `ExecutionFailed()` error when the Safe transaction fails, instead of bubbling up the original revert reason from `execTransactionFromModuleReturnData()`:

src/DelegatorModule.sol:L143-L147

```
function _execute(address _target, uint256 _value, bytes calldata _callData) internal returns (bytes memory returnData_) {
    (bool success, bytes memory returnData) =
        ISafe(safe()).execTransactionFromModuleReturnData(_target, _value, _callData, Enum.Operation.Call);
    if (!success) revert ExecutionFailed();
    return returnData;
```

This behavior hides information that could help diagnose why the underlying transaction failed. It is also very common (e.g., standard Solidity behavior) to propagate the revert reason.

Recommendation

The Delegation Framework still uses OpenZeppelin Contracts v4, whose `Address.verifyCallResult()` function (here) essentially offers this functionality. However, if the return data from the call is empty (i.e., there is nothing to bubble up), it reverts with an old-style string error (that has to be provided as argument to the function), instead of a custom error. That doesn't fit very well into this module, which uses custom errors.

The `Address.verifyCallResult()` function from OpenZeppelin Contracts v5 (here) uses a custom error (defined in OZ) instead of a string error and would therefore be more appropriate for the module. We suggest utilizing that version of the function or – if using OZ v5 in the module, while the Delegation Framework uses v4, is deemed unsatisfactory – similar self-written code for the time being.

5.7 Misleading Behavior of Calls Made Directly on the Implementation Contract Minor ✓ Fixed

Resolution

All external/public functions now have an `onlyProxy` modifier, which reverts for calls made directly on the implementation contract. The previously public `safe()` function (which was also used internally) has been split up into an internal and an external version to avoid repeating `onlyProxy` checks in the same call. (More precisely, `safe()` is now external, and the internal function `_getSafeAddressFromArgs()` has been renamed to `_safe()` and is now used to retrieve the Safe's address internally.) The contract's NatSpec has been improved as recommended.

Description

The `DelegatorModule` contract is designed to be used as an implementation contract for proxies, where each proxy instance is associated with a specific Safe. However, some functions, when called directly on the implementation, show irritating or misleading behavior.

Examples

- The `safe()` function returns a slice of 20 bytes from the contract's own bytecode. In the case of a proxy, this is indeed the Safe address with which the module is associated, but for the implementation, it is just some 20 bytes that will almost certainly not be an address at which a Safe is deployed. While we don't see any serious problem resulting from this (not even if the bytecode could be specifically crafted to include a particular address at this location), it can still be considered misleading if calling `safe()` on the implementation returns a "strange" address that is not a Safe.
- The `supportsInterface()` function returns true for the three interfaces the module supports – even if called on the implementation contract. The implementation contract is not functional when used directly, though, so it should better not claim to support any interfaces at all.

Recommendation

While, as mentioned above, we don't see any serious problems coming out of this, the clean behavior would be for all external and public functions to revert with a dedicated error when called directly on the implementation contract. This can be achieved

with a modifier `revertOnImplementation` that compares `address(this)` to the deployment address of the implementation contract. The latter is available in the constructor as `address(this)` and can be stored in an immutable variable. If the two are the same, the implementation contract was called directly. The downside of this is additional code and slightly increased gas consumption for legitimate calls on the proxy. Compared to the heavy lifting in the Delegation Manager, the latter seems negligible, though.

That said, if a module proxy is accidentally deployed with an address that is not a Safe, this proxy exhibits the same behavior as the implementation does right now, and it is unclear how far one wants to go to try to avoid such situations…

In any case, it might make sense to add to the NatSpec of `DelegatorModule` that all modules must be deployed through the accompanying `DelegatorModuleFactory` and that the implementation contract itself, as well as all other direct deployments of this contract, are not usable as module.

5.8 Solidity Version Pragma Allows Outdated Compiler Versions Minor ✓ Fixed

Resolution

For consistency with the Delegation Framework, the version pragma has been changed to `0.8.23`.

Description

The contracts specify Solidity version `^0.8.13`, whereas the current stable release is `0.8.30` from May 2025, and all versions older than `0.8.23` have known issues. The `foundry.toml` file doesn't enforce a specific compiler version (or any compiler-related settings, for that matter). What foundry ends up using is 0.8.23, so the contracts are compiled with a version without known issues. That said, we don't see a convincing reason to specify a version pragma that would even *allow* compiling the contracts with a version that has known bugs.

Recommendation

We think more sensible choices for the version pragma are:

1. `^0.8.23`: The version specified in the delegation framework is `0.8.23` (non-floating), and we shouldn't fall behind that.
2. `0.8.23`: That would be the exact same version pragma as in the delegation framework.
3. `^0.8.30`: The most recent release from the 0.8 series. (Still not too recent.) Also allows newer versions, once released.
4. `0.8.30`: In case the (currently) most recent version should be used but newer versions, when released, shouldn't be allowed.

Moreover, consider adding compiler-related settings to `foundry.toml`, such as the specific compiler version to be used, possibly optimizations and target EVM, etc.

5.9 Module Emits No Events ✓ Fixed

Resolution

Client's comment:

> Event emission is delegated to the `DelegationManager`. We intentionally do not emit events within the execution functions, as this aligns with the delegation framework's design.

Since not emitting events is a conscious decision, we consider this fixed.

Description and Recommendation

There is only one non-view external or public function in `DelegatorModule`: `executeFromExecutor()`. This function doesn't emit any events, nor do the internal functions that are called during its execution. It might very well be appropriate to rely solely on the events emitted by the Safe and the Delegation Manager, especially since the module is stateless and only passes the execution request through, but if these are considered insufficient, an event emission could be added to `executeFromExecutor()`.

Remark

If, as a result of issue 5.1, new non-view functions are introduced, these should be taken into account here too and treated accordingly.

5.10 Code Triggers Some Compiler Warnings ✓ Fixed

Resolution

Client's comment regarding the "immutables should use SCREAMING_SNAKE_CASE" warnings:

> The `DeleGatorModule` follows camelCase naming for consistency with the delegation framework.

All other warnings have been addressed.

Description

The compiler generates the following warnings:

```
Compiler run successful with warnings:
Warning (2018): Function state mutability can be restricted to pure
  --> test/mocks/MockSafe.sol:90:5:
   |
90 |     function isValidSignature(bytes32, bytes memory) external view returns (bytes4 magicValue) {
   |     ^ (Relevant source part starts here and spans across multiple lines).

note[screaming-snake-case-immutable]: immutables should use SCREAMING_SNAKE_CASE
  --> [...]/delegator-safe-module/src/DelegatorModule.sol:24:30
   |
24 |     address public immutable delegationManager;
   |                              -----------------
   |
   = help: https://book.getfoundry.sh/reference/forge/forge-lint#screaming-snake-case-immutable

note[unused-import]: unused imports should be removed
 --> [...]/delegator-safe-module/script/DeployDelegatorModule.s.sol:5:10
  |
5 | import { DelegatorModule } from "../src/DelegatorModule.sol";
  |          ---------------
  |
  = help: https://book.getfoundry.sh/reference/forge/forge-lint#unused-import

note[screaming-snake-case-immutable]: immutables should use SCREAMING_SNAKE_CASE
  --> [...]/delegator-safe-module/src/DelegatorModuleFactory.sol:12:30
   |
12 |     address public immutable implementation;
   |                              --------------
   |
   = help: https://book.getfoundry.sh/reference/forge/forge-lint#screaming-snake-case-immutable

note[screaming-snake-case-immutable]: immutables should use SCREAMING_SNAKE_CASE
  --> [...]/delegator-safe-module/src/DelegatorModuleFactory.sol:13:30
   |
13 |     address public immutable delegationManager;
   |                              -----------------
   |
   = help: https://book.getfoundry.sh/reference/forge/forge-lint#screaming-snake-case-immutable
```

Recommendation

Consider addressing these warnings to improve code quality and comply with existing conventions.

Appendix 1 - Files in Scope

This review covered the following files.

Initial version at revision `77dae3ffe31df1786ce633a1b510d3c31f922bd5`:

File	SHA-1 hash
src/DelegatorModule.sol	`2bb729a687c33285269a2aaa59c5a895b63ebd76`
src/DelegatorModuleFactory.sol	`bc3da9b1e89413ae6ef94c3cd15c71767d175f19`

Final version at revision `b61e1e4adad1526f970d6b39469e031b4e7ac2a2`:

File	SHA-1 hash
src/DeleGatorModule.sol	`8c59be2c63f383a876bb7d7a9c16f1eec09d7dcc`
src/DeleGatorModuleFactory.sol	`f081b622e38e150f11747b0924a3f8db4ea6f81e`

Appendix 2 - Disclosure

Consensys Diligence (“CD”) typically receives compensation from one or more clients (the “Clients”) for performing the analysis contained in these reports (the “Reports”). The Reports may be distributed through other means, including via Consensys publications and other distributions.

The Reports are not an endorsement or indictment of any particular project or team, and the Reports do not guarantee the security of any particular project. This Report does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. No Report provides any warranty or representation to any third party in any respect, including regarding the bug-free nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the Reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset. Specifically, for the avoidance of doubt, this Report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. CD owes no duty to any third party by virtue of publishing these Reports.

A.2.1 Purpose of Reports

The Reports and the analysis described therein are created solely for Clients and published with their consent. The scope of our review is limited to a review of code and only the code we note as being within the scope of our review within this report. Any Solidity code itself presents unique and unquantifiable risks as the Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond specified code that could present security risks. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. In some instances, we may perform penetration testing or infrastructure assessments depending on the scope of the particular engagement.

CD makes the Reports available to parties other than the Clients (i.e., “third parties”) on its website. CD hopes that by making these analyses publicly available, it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.

A.2.2 Links to Other Web Sites from This Web Site

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Consensys and CD. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Consensys and CD are not responsible for the content or operation of such Web sites, and that Consensys and CD shall have no liability to you or any other person or entity for the use of third party Web sites. Except as described below, a hyperlink from this web Site to another web site does not imply or mean that Consensys and CD endorses the content on that Web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the Reports. Consensys and CD assumes no responsibility for the use of third-party software on the Web Site and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.

A.2.3 Timeliness of Content

The content contained in the Reports is current as of the date appearing on the Report and is subject to change without notice unless indicated otherwise, by Consensys and CD.